

ML System Design and MLOps Foundations

A production ML system includes the processes that create, validate, serve, and monitor predictions. The weights are one component. Define feature availability, label delay, version compatibility, latency, and recovery so a statistically promising model can make dependable decisions in the real application.

For: ML and backend candidates who want to connect models to reliable data pipelines, point-in-time features, serving, experiments, monitoring, and rollback.

Suggested pace: 4 weeks after basic ML evaluation | Intermediate engineer

What you will be able to do

- Design separate training and serving paths with compatible data contracts.
- Prevent leakage through point-in-time feature construction.
- Plan model rollout, fallback, monitoring, and rollback.
- Explain cold start, delayed labels, drift, and feedback loops in a concrete product.

Build the right mental model

Time is part of a feature

A feature needs an entity key, event time, availability time, version, and freshness policy. A value describing an old event may still have arrived too late for a historical prediction. Point-in-time correctness must respect what was actually available then.

Version the whole decision path

Model, preprocessing, feature definitions, label logic, and serving configuration can change independently. Record compatible versions and a rollback path. A model registry without data and feature provenance leaves important behavior unexplained.

Monitor more than output distribution

Track data quality, missing features, freshness, latency, errors, prediction distribution, and mature outcome metrics. A drift alert is a reason to investigate, not an automatic instruction to retrain. Upstream bugs and policy changes may require different fixes.

Study with evidence.

Use a small recommendation or risk-scoring case with synthetic data. State the objective and baseline, then walk training, online serving, feedback, and failure recovery. Keep offline quality separate from online causal impact and service reliability.

Week 1: Frame and collect

Define prediction unit, label window, exposure data, and a baseline. Establish temporal or grouped evaluation and record data provenance. For recommendations, distinguish an item not clicked from one never shown.

Deliverable and check: A task contract and dataset design that does not leak future outcomes or confuse exposure with preference.

Week 2: Features and training

Create point-in-time features, version transformations, and validate missing or delayed inputs. Train a simple model and compare it fairly with the baseline. Record hyperparameters, code, and evaluation artifacts.

Deliverable and check: A reproducible training run and an explicit feature-availability boundary.

Week 3: Serving and rollout

Define API latency, model/feature compatibility, fallbacks, shadow or canary evaluation, and rollback. Estimate online retrieval and ranking work separately. Test a missing feature store and a slow model instance.

Deliverable and check: A serving path with bounded resources and a clear degraded-mode contract.

Week 4: Feedback and operations

Design delayed-label evaluation, drift investigation, and experiment guardrails. Review cold-start cohorts and feedback loops. Practice one incident that requires data repair rather than model retraining.

Deliverable and check: A full system-design walkthrough covering quality, service behavior, and recovery.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Prevent a feature pipeline from leaking the future

A fraud model uses a customer's rolling transaction total. Historical training rows are joined to today's latest total, which includes transactions that occurred after the historical prediction time. Offline performance looks unusually strong.

A worked approach

1. Define each prediction timestamp and the feature's event and availability times. Rebuild the aggregate using only records that would have been visible at that moment, including realistic ingestion delays where relevant.
2. Use an as-of or point-in-time join with versioned feature logic and a temporal evaluation split. Compare the corrected baseline and model with the leaked version, expecting the inflated score to fall rather than treating that as a regression to hide.
3. Align online serving with the same transformation semantics and monitor freshness. Add a test containing a future transaction that must not change an earlier row's features. Preserve the data and code version used for each evaluation.

Expected result

The corrected evaluation represents a feasible deployment decision. The system can explain feature provenance and avoids selecting a model whose apparent advantage came from unavailable information.

Verify the result

- Does adding a future event leave past features unchanged?
- Are training and serving transformations compatible?
- Can delayed labels and late-arriving data be distinguished?

Extend the exercise

Simulate a feature-store outage and compare a defined stale-feature fallback with a simpler baseline model. Explain which decisions can tolerate degraded inputs and which must fail explicitly.

Make the learning stick.

What is training-serving skew?

It is a mismatch between features, transformations, or behavior used during training and those used online. Shared definitions, versioning, point-in-time validation, and monitoring reduce the risk but still need concrete tests.

Should every drift alert trigger retraining?

No. Investigate whether the cause is data corruption, a feature bug, changed traffic, delayed labels, or genuine distribution change. Retraining on broken data can amplify the problem rather than repair it.

Why can a recommendation model create feedback loops?

Its choices influence what users see and therefore what labels are collected. Without exposure logging, appropriate exploration, and careful evaluation, the system may reinforce its own past selections and hide alternatives.

Avoid these traps

- Joining historical examples to the latest feature snapshot.
- Versioning only model weights and ignoring transformations.
- Equating offline metric improvement with safe online rollout.

Ready for the next step?

- Feature and label timing are explicit.
- Training and serving contracts are version-compatible.
- Rollout and fallback behavior are tested.
- Monitoring distinguishes data, model, and service failures.

Reliable next reads

- [Rules of Machine Learning](#)
- [Google recommendation systems](#)
- [Azure AI/ML architecture](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.